ShortP2P Protocol

（译文可能存在不准确之处；以英文技术说明与源代码为准。）

(Translation may contain inaccuracies; the English technical description and the source code are authoritative.)

语言: Chinese Simplified / 简体中文

Author: Igor Evdokimov

Repository: https://github.com/IskraMmessenger/ShortP2P

Created: 2026-11-11

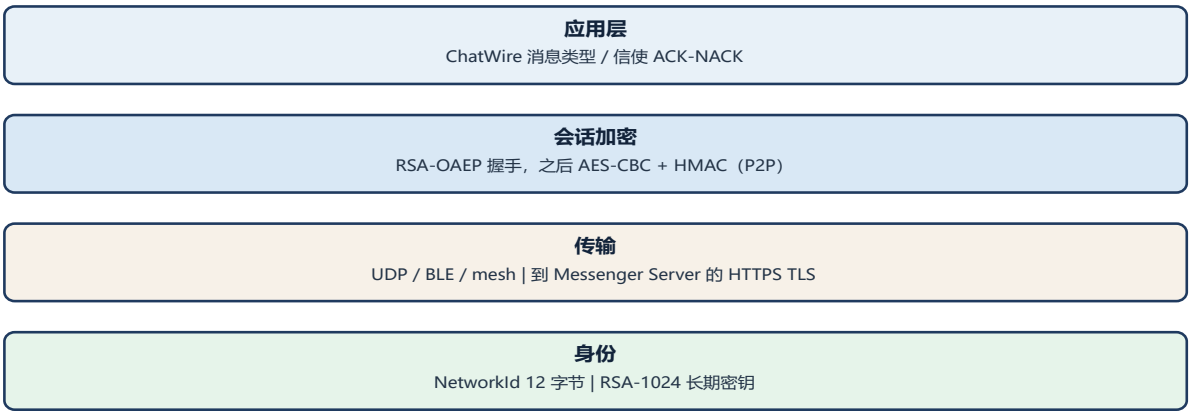
Modified: 2026-11-11

Related: docs/encryption, docs/messages, protocol.md

目录
封面
目录
通用图示
协议说明

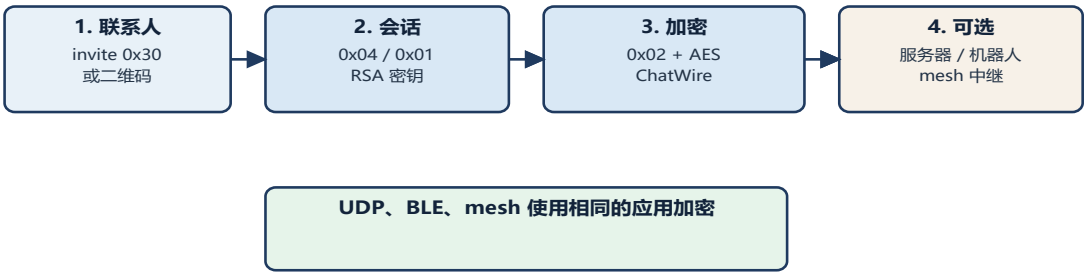
通用图示

协议分层

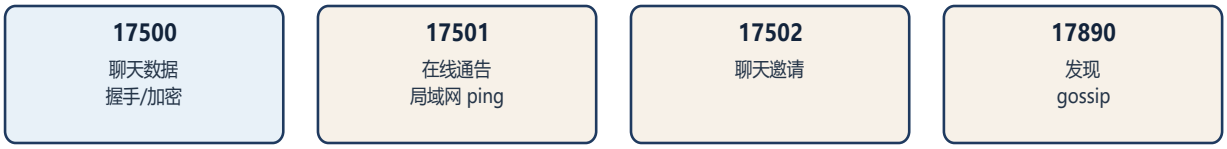


逻辑栈（非操作系统层）。详见 docs/encryption、docs/messages。

典型直连聊天流程



默认 **UDP** 端口



蓝牙与 mesh 复用相同逻辑帧；Messenger Server 使用 HTTPS。

协议说明

ShortP2P 是一个点对点消息协议栈。通俗地说：两台设备（对等端）互相找到对方，协商出秘密密钥，
之后交换任何中间人都无法读取的聊天消息。传输可以直接进行（UDP、蓝牙、mesh/LoRa 无线电），
也可以经由可选的 Messenger Server 中转——它只转发已经加密的数据。

基础概念（无需任何密码学知识）

- 对等端（peer）：与你聊天的另一台设备或人。
- NetworkId：12 个随机字节，用来标识一个用户，类似于网络地址。在界面和二维码中以短 base64url 文本（约 16 个字符）显示。
- RSA-1024：一种非对称算法——一对密钥。公钥可以公开传递（在邀请和二维码中）；私钥永不离开设备。用公钥加密的内容只有对应的私钥才能解密。在 ShortP2P 中，它每个会话只用一次，用来把会话密钥发给对方。
- 会话密钥：为一次聊天会话随机生成的两串字节：AES 密钥（16 字节）和 MAC 密钥（32 字节）。握手之后，双方持有相同的两串密钥。
- AES-128-CBC：一种对称加密——同一个密钥既加密也解密。CBC 模式把块链接起来，使相同的明文片段看起来不同；PKCS7 填充把长度补齐到 16 字节的倍数。
- IV（初始向量）：每个包 16 个全新的随机字节，使同一段文本每次加密的结果都不一样。
- HMAC-SHA256 标签：带密钥的校验和（截取 16 字节），对 IV+密文计算。接收方重新计算并以恒定时间比较；不一致说明包被篡改或损坏，直接丢弃。
- ChatWire：ShortP2P 自有的聊天内容二进制格式。被加密并发送的就是它；接收方先解密，再解析。
- S2P1：每个 ChatWire 帧开头的 4 字节“魔数签名”（ASCII 字符 "S2P1"）。它像文件头一样工作——解析器最先检查它来识别格式。
- 类型字节（kind）：紧跟魔数之后的那个字节，它的值说明后面是哪种消息。
- ACK / NACK：内部的投递回执。ACK = “消息已完整收到”；NACK = “请重发这些编号的分块”。
- TLS/HTTPS：客户端与 Messenger Server 之间标准的传输加密（浏览器里那把锁）。
- JWT：登录时服务器签发的令牌。客户端在每个请求中附带它；服务器验证签名，而不必再要密码。
- 端到端加密（E2E）：只有两台聊天设备能读内容。Messenger Server 只存储并转发不透明的加密字节块（Base64 文本），自己也读不了。
- 机器人（bot）：服务器上的程序账号，客户端可以与它对话；它同样只能收到加密内容。

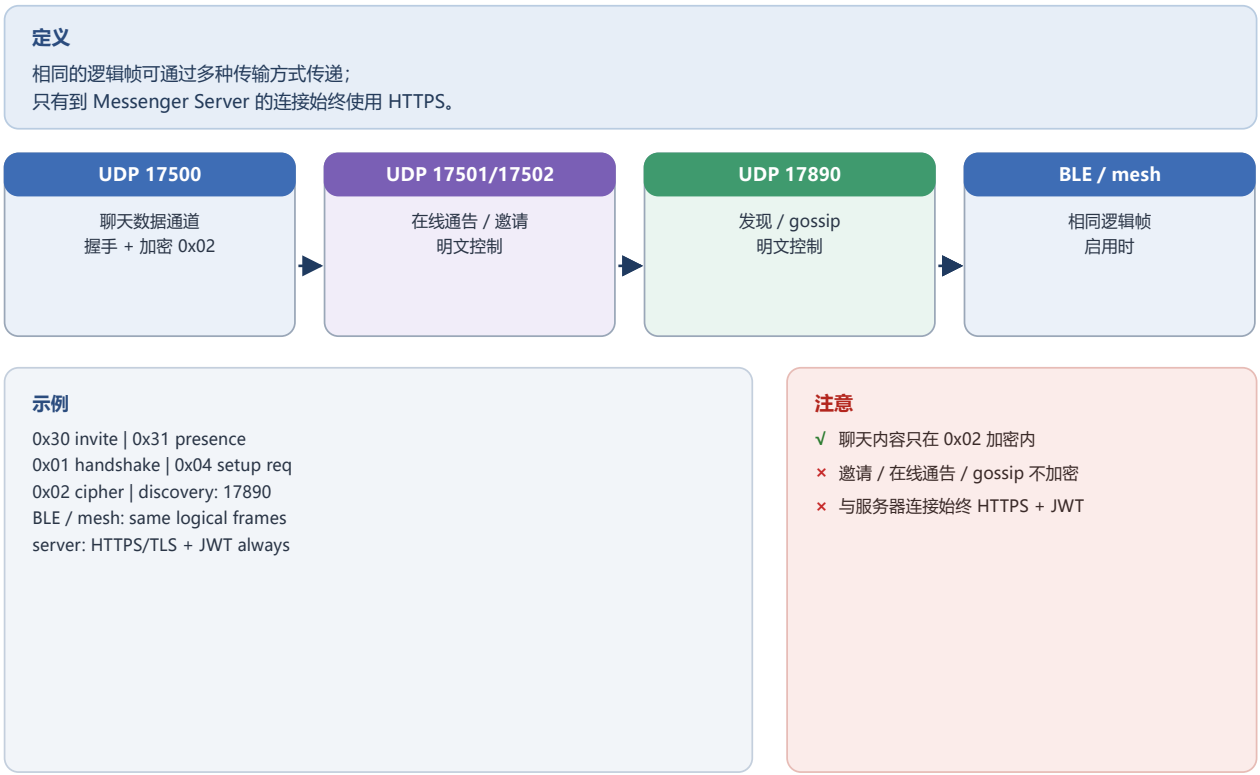
- Gossip / discovery：回答“谁在附近、如何连到它”的辅助流量（UDP 17890）。
- RELAY：一种局域网路由帧，可以携带已加密的数据穿过中间节点。

1) 身份



每个用户有一个 NetworkId（12 个随机字节），以及注册时生成的 RSA-1024 密钥对。公钥以明文形式随聊天邀请（0x30）或二维码传递——这是安全的，公钥本来就是用来公开的。规则：NetworkId 较小的一方成为握手“领导者”，另一方等待。

2) 传输



默认 UDP 端口：17500——聊天数据通道（握手与加密包）；17501——在线通告；17502——聊天邀请；17890——发现/gossip。启用时，蓝牙与 mesh/LoRa 承载相同的逻辑帧。与 Messenger Server 的通信始终走 HTTPS/TLS 并使用 JWT 认证。

3) 会话（直连 P2P），逐步说明



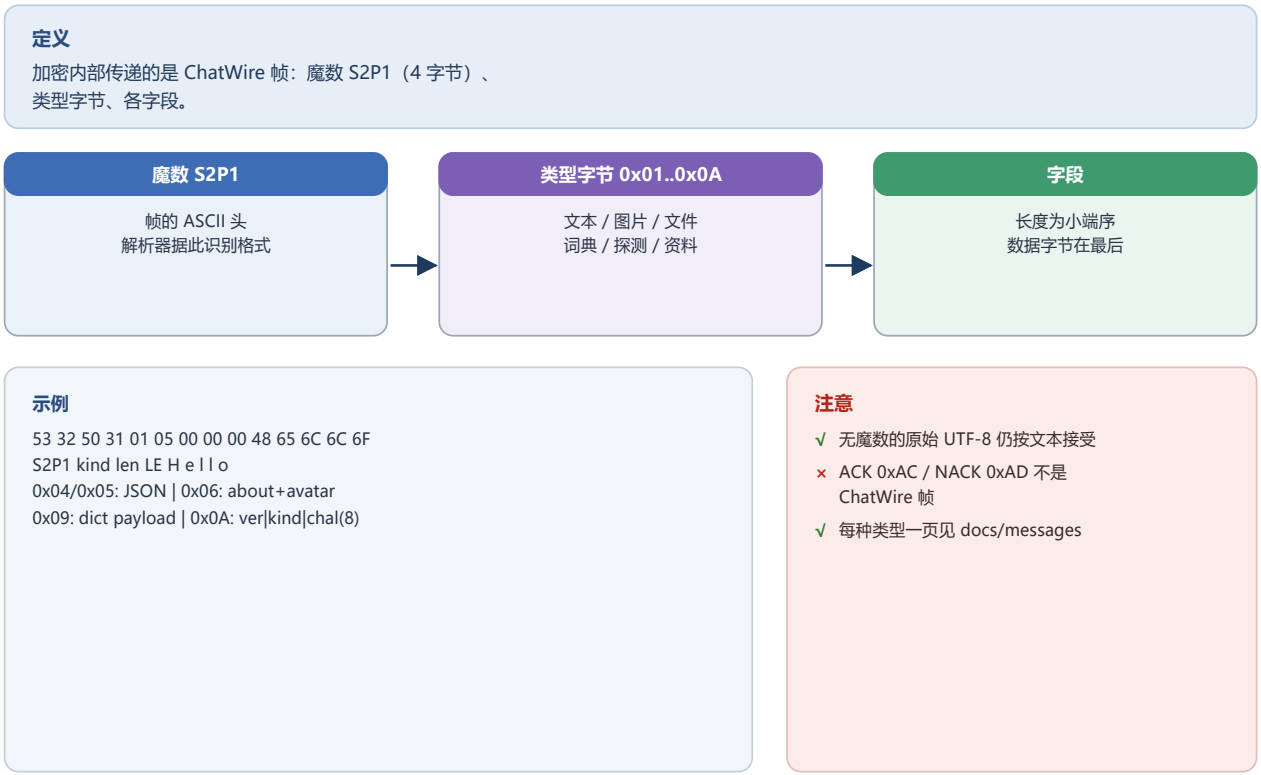
a. 跟随者可以发送带自己 NetworkId 的 0x04 帧：“我想要一个会话”。

b. 领导者生成两个随机密钥（AES 密钥 16 字节 + MAC 密钥 32 字节），用对方的 RSA 公钥以 RSA-OAEP(SHA-1) 加密这 48 字节。结果正好 128 字节，作为 0x01 帧发送。

c. 跟随者用自己的 RSA 私钥解密；此后双方共享同样的两个密钥。

d. 之后每个数据帧都是 0x02 + 包，包 = IV(16) + AES-CBC 密文 + HMAC 标签(16)，最大 1024 字节。接收方先验证标签，再解密。

4) 应用消息（加密内部）



送进加密的明文通常是一个 ChatWire 帧：魔数 "S2P1"，然后是类型字节，然后是各字段（长度除特别说明外均为小端序）。

类型字节的取值，逐一说明：

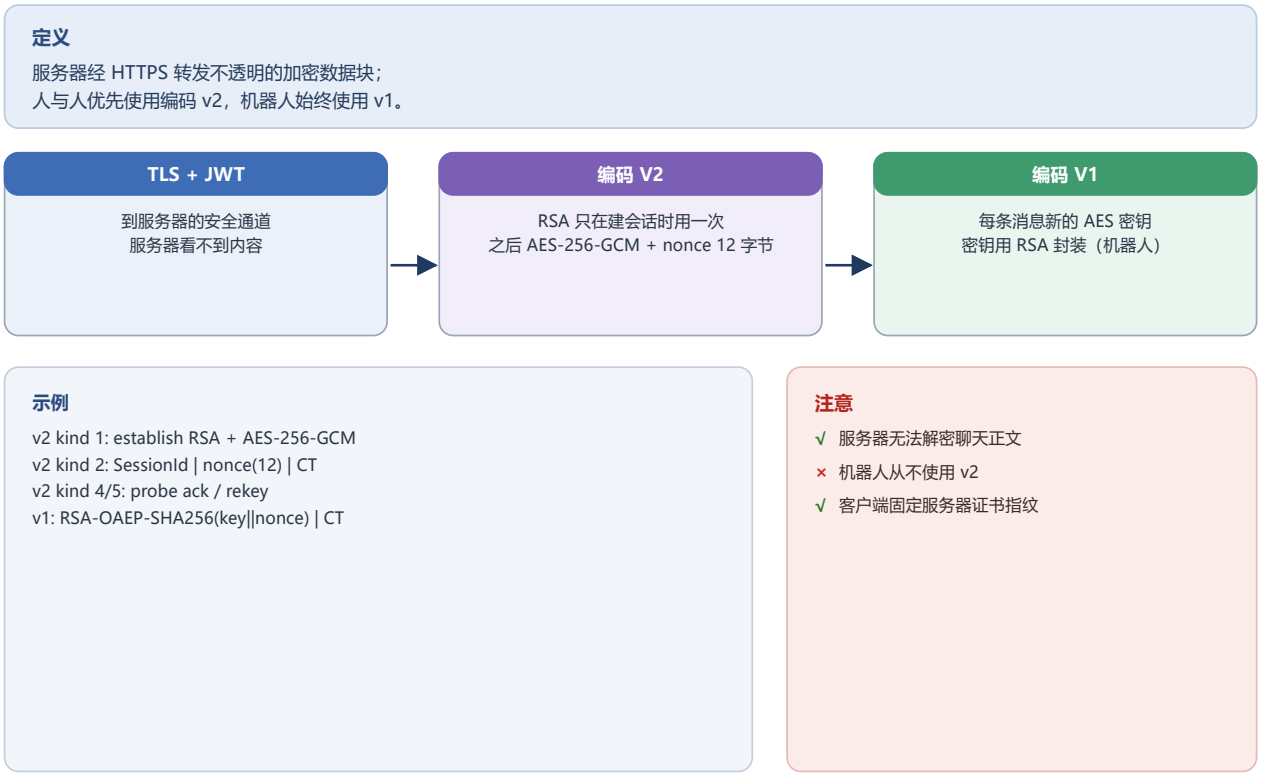
- 0x01 Text——普通的文字消息（UTF-8），即人输入的内容。
- 0x02 Image——聊天中的内嵌图片：MIME 类型（如 "image/png"）加上图片字节。
- 0x03 File——随聊天发送的小文件：文件名、MIME 类型、字节。
- 0x04 Transfer offer——一份 JSON 描述：一个大文件在聊天之外下载（连接哪个主机/端口、一次性令牌、大小、有效期）。
- 0x05 Transfer control——对 offer 的 JSON 回复：接受、拒绝或错误。
- 0x06 User info——发送者的资料（“关于我”文字和头像），不显示为聊天气泡。
- 0x07 Vocabulary offer——发送者支持的压缩词典列表。

- 0x08 Vocabulary accept——本次聊天选定的词典（或 "UNCOMPRESSED"）。
- 0x09 Compressed text——用选定词典编码的文字，节省流量（适合慢速无线链路）。
- 0x0A Session probe——穿过加密信道的小“ping”，确认双方都能正确解密。

不是 ChatWire 帧、但同样加密发送的：0xAC ACK（消息已完整收到，17 字节）和 0xAD NACK（请重发列出的分块编号）。很旧的客户端发送不带 S2P1 魔数的原始 UTF-8；接收方仍按文本处理。

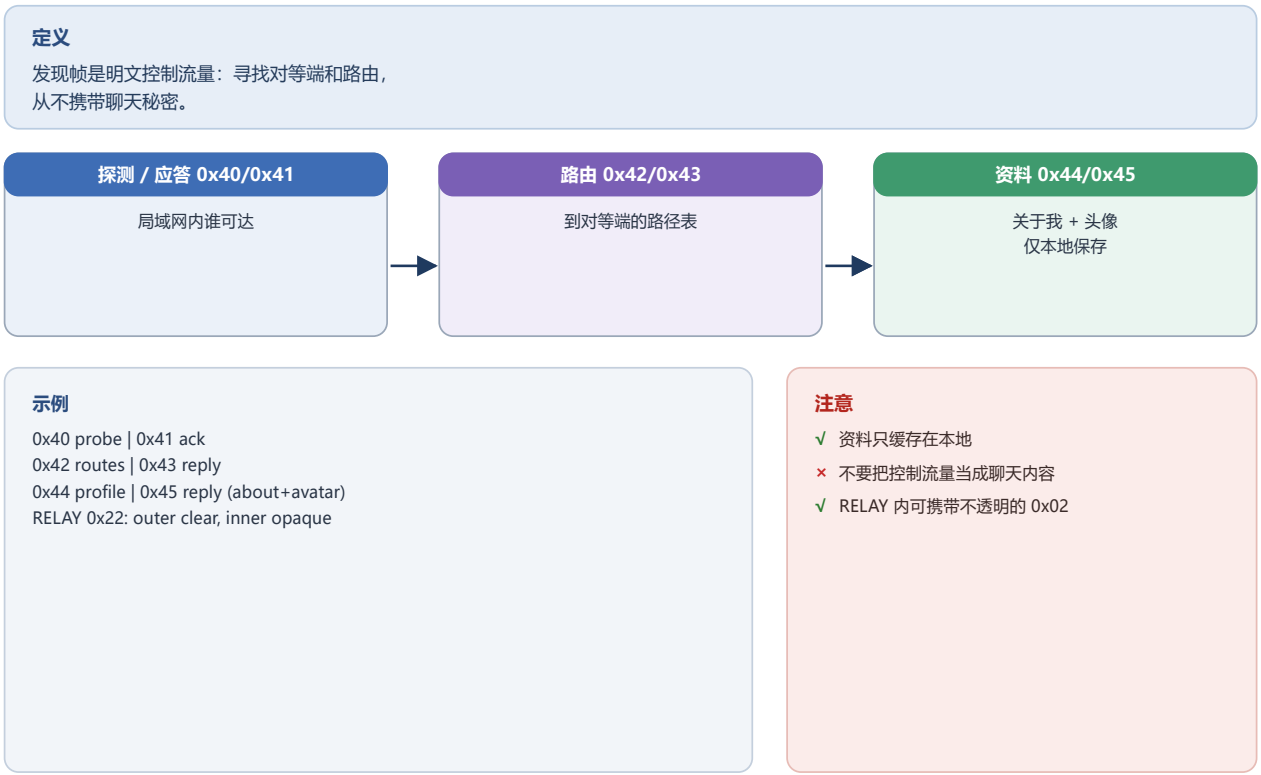
每种消息一页、含字节布局与未加密示例：docs/messages。加密流程图：docs/encryption。

5) Messenger Server 与机器人



当双方无法直连时，走 Messenger Server（HTTPS/TLS+JWT）。聊天正文是端到端加密的，服务器只保存不透明的 Base64 字节块。
- 编码 v2（人与人之间首选）：RSA 只在建会话时用一次；之后每条消息用 AES-256-GCM（一种认证加密：一个算法同时完成加密和完整性校验），每条消息一个 12 字节随机 nonce（一次性随机数）。
- 编码 v1（用于机器人及后备）：每条消息都生成新的 AES-256-GCM 密钥和 nonce，并用接收者的 RSA 公钥包裹密钥+nonce（RSA-OAEP-SHA256）。机器人始终用 v1。
两种情况下服务器都无法解密内容。

6) 发现



UDP 17890 上的 0x40..0x45 帧（探测/应答、路由表、资料）大多是明文控制流量：用于寻找对等端，不要与聊天的秘密内容混淆。局域网 FIND/FOUND 帧找到对方地址；RELAY 帧可以携带已加密的 0x02 负载穿过中间节点。

延伸阅读

- docs/encryption——以矢量图展示的加密流程（会话建立、包格式、服务器路径）。
- docs/messages——每种消息一页，含未加密的字节级示例。
- protocol.md——详细的帧参考（俄文）。